Web Dev Mastery

Complete Guide on CSS Selectors with Examples

Topics Covered:

1. **Element Selector**
2. **Class Selector**
3. **ID Selector**
4. **Child Selector**
5. **Descendant Selector**
6. **Universal Selector**
7. **Pseudo-Selectors**
    - `:first-child`
    - `:last-child`
    - `:nth-child(n)`
    - Anchor States (`a:link`, `a:visited`, `a:hover`, `a:active`)
8. **Pseudo-elements**
    - `::before`
    - `::after`

---

1. Element Selector

Explanation:

The **element selector** selects all elements of a specific HTML tag type. It is one of the most basic selectors and applies styles universally to elements of the same type, such as `<div>`, `<p>`, `<h1>`, etc.

Use Case:

You would use this selector when you want to apply a style to every instance of a certain element on the page. For example, if you want all paragraphs or all `<div>` elements to have a consistent style.

Example:

HTML:

```
<div>This is a div</div>
<p>This is a paragraph</p>
```

CSS:

```
div {
  background-color: lightblue;
  padding: 10px;
}

p {
  color: green;
}
```

---

2. Class Selector (`.className`)

Explanation:

The **class selector** selects all elements that have a specific class. Multiple elements can share the same class, making this selector ideal for styling groups of elements.

Use Case:

This selector is used to apply styles to a group of elements that share a similar class, such as all buttons, headers, or any other grouping of HTML elements that require consistent styling.

Example:

HTML:

```
<div class="box">Box 1</div>
<div class="box">Box 2</div>
```

CSS:

```
.box {
  border: 2px solid black;
  margin: 10px;
  padding: 10px;
}
```

---

3. ID Selector (`#id`)

Explanation:

The **ID selector** is used to target a single element with a specific `id`. IDs should be unique in a document, so only one element should have a particular ID.

Use Case:

This selector is used when you need to apply a unique style to one element, like a specific button or container. For example, styling the header or footer of a page.

Example:

HTML:

```
<div id="unique">Only One Element</div>
```

CSS:

```
#unique {
  background-color: yellow;
  font-weight: bold;
}
```

---

4. Child Selector (`parent > child`)

Explanation:

The **child selector** targets **direct** children of a specified parent element. This means it only selects the immediate children, not deeper descendants.

Use Case:

It is used when you want to style only the direct children of a parent and exclude any deeper nested children.

Example:

HTML:

```
<div class="parent">
  <p>Child 1</p>
  <span>Child 2</span>
  <div>
    <p>Nested Child (won’t be selected)</p>
  </div>
</div>
```

CSS:

```
.parent > p {
  color: red;
}
```

---

5. Descendant Selector (`parent child`)

Explanation:

The **descendant selector** targets all descendants of a specific parent, regardless of how deeply nested they are.

Use Case:

You use this selector when you want to apply styles to all child elements, even if they are nested within other elements.

Example:

HTML:

```
<div class="ancestor">
  <div>
    <p>This will be selected</p>
  </div>
</div>
```

CSS:

```
.ancestor p {
  font-style: italic;
}
```

---

6. Universal Selector (*)

Explanation:

The **universal selector** selects every element on the page. This is often used to apply a global style to all elements or for resetting margins, padding, etc.

Use Case:

It's useful when you want to apply a universal style, like resetting the margin and padding for all elements to ensure consistency across browsers.

Example:

HTML:

```
<div>Universal Styling</div>
<p>Applies to all</p>
```

CSS:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

---

7. Pseudo-Selectors

7.1. `:first-child`

Explanation:

This pseudo-class targets the **first child** element of a parent.

Use Case:

You can use `:first-child` to style the first element in a container, such as the first list item or paragraph.

Example:

HTML:

```
<ul>
  <li>First</li>
  <li>Second</li>
</ul>
```

CSS:

```
li:first-child {
  color: blue;
}
```

7.2. `:last-child`

Explanation:

This pseudo-class targets the **last child** of a parent.

Use Case:

You can use `:last-child` to apply styles to the last item in a list or container.

Example:

CSS:

```
li:last-child {
  color: red;
}
```

7.3. `:nth-child(n)`

Explanation:

The **nth-child** selector allows you to select elements based on their **position** in a parent. `n` can be a number or a formula like `2n` to target every second element.

Use Case:

This is useful for styling specific items, such as every other item in a list or grid.

Example:

CSS:

```
li:nth-child(2) {
  color: green;
}
```

7.4. Anchor States

Explanation:

These pseudo-classes style `<a>` elements based on their **state** (normal, visited, hovered, or active).

Use Case:

You can style links based on whether they've been clicked, hovered over, or are in their normal or active state.

Example:

HTML:

```
<a href="#">Normal Link</a>
<a href="#">Visited Link</a>
<a href="#">Hover Link</a>
<a href="#">Active Link</a>
```

CSS:

```
a:link {
  color: blue;
```

```
}

a:visited {
  color: purple;
}

a:hover {
  color: orange;
  text-decoration: underline;
}

a:active {
  color: red;
}
```

---

8. Pseudo-elements

8.1. `::before` and `::after`

Explanation:

The **::before** and **::after** pseudo-elements allow you to **insert content** before or after an element's content.

Use Case:

These are useful for adding extra content (e.g., icons or text) to elements without modifying the HTML.

Example:

HTML:

```
<p class="fancy">Hello</p>
```

CSS:

```
.fancy::before {
  content: "★ ";
```

```
}
.fancy::after {
  content: " ★";
}
```

Summary of CSS Selectors:

1. **Element Selector** targets specific HTML elements (e.g., `<div>`, `<p>`).
2. **Class Selector** applies styles to all elements with a given class.
3. **ID Selector** targets a single, unique element.
4. **Child Selector** targets direct children of a parent.
5. **Descendant Selector** targets all descendants, no matter how deeply nested.
6. **Universal Selector** applies styles to all elements.
7. **Pseudo-Selectors** are used for targeting elements in specific states or positions (like `:first-child`, `:last-child`, `:nth-child`).
8. **Anchor States** (`a:link`, `a:visited`, `a:hover`, `a:active`) style anchor tags based on their state.
9. **Pseudo-elements** (`::before`, `::after`) allow you to insert content before or after elements without modifying HTML.

Conclusion:

This guide covers the most essential CSS selectors used for styling HTML elements efficiently. Understanding these selectors will allow you to create dynamic, responsive, and visually appealing web pages.